



CP Bootcamp 2026 — Day 7

Theme: Searching + Binary Search Foundation

Main Goal:

Data-এর মধ্যে কিছু খোঁজার সময় brute force থেকে efficient thinking-এ যাওয়া।

আগে:

Traverse Everything



Check One by One

এখন:

Understand Search Space



Remove Impossible Parts



Search Faster



Day 7 Chapter Map

Day 7

- Chapter 1 – Searching Mental Model
- Chapter 2 – Linear Search Pattern
- Chapter 3 – Linear Search Variations
- Chapter 4 – Search State (Found, Index, Count)
- Chapter 5 – Sorted Data & Search
- Chapter 6 – Binary Search Mental Model
- Chapter 7 – Binary Search Implementation
- Chapter 8 – Binary Search Patterns
- Chapter 9 – Common Bugs & Edge Cases
- Chapter 10 – Day 7 Assignment & Practice

Chapter 1 — Searching Mental Model

Search কী?

Search মানে:

কোনো collection-এর ভিতরে নির্দিষ্ট information খুঁজে বের করা।

Example:

Array:

4 8 2 7 9

Find:

7

Question:

7 আছে কি?

কোথায় আছে?

কতবার আছে?

Search Problem-এর ৩টি প্রশ্ন

যেকোনো search problem দেখলে:

Question 1

কি খুঁজছি?

Example:

Value

Index

Count

Position

Minimum

Maximum

Question 2

Data sorted কিনা?

Example:

Sorted:

1 3 5 7 9

Unsorted:

7 2 9 1 5

Sorted হলে extra information পাওয়া যায়।

Question 3

আমাকে সব জায়গা check করতে হবে?

নাকি:

Half বাদ দেওয়া যাবে?

এখান থেকেই Binary Search আসে।

Chapter 2 — Linear Search Pattern

Linear Search কী?

সব element একে একে check করা।

Example:

Array:

5 8 3 9 2

Find:

9

Process:

5 → No

8 → No

3 → No

9 → Found

Mental Model

Start

↓

Check Current Element

↓

Match?

├ Yes → Answer
└ No → Next Element

Basic Template

```
for(int i = 0; i < n; i++)  
{  
    if(arr[i] == target)  
    {  
        // found  
    }  
}
```

Example

```
#include <stdio.h>

int main()
{
    int arr[] = {5,8,3,9,2};

    int n = 5;
    int target = 9;

    int found = 0;

    for(int i=0;i<n;i++)
    {
        if(arr[i]==target)
        {
            found = 1;
            break;
        }
    }

    if(found)
        printf("Found");
    else
        printf("Not Found");

    return 0;
}
```

Complexity

Array size:

n

Worst case:

সব element check করতে হবে।

$O(n)$

Chapter 3 — Linear Search Variations

Linear Search শুধু "আছে কিনা" না।

অনেক variation আছে।

1. Find Index

Problem:

Target কোথায় আছে?

Example:

5 8 3 9

Target:

3

Answer:

Index = 2

Code idea:

```
int index = -1;

for(int i=0;i<n;i++)
{
    if(arr[i]==target)
    {
        index=i;
        break;
    }
}
```

2. Count Occurrence

Problem:

কতবার এসেছে?

Example:

5 2 5 3 5

Target:

5

Answer:

3

Pattern:

Search

+

Counter

Code:

```
int count=0;

for(int i=0;i<n;i++)
{
    if(arr[i]==target)
    {
        count++;
    }
}
```

3. First Occurrence

Example:

2 5 3 5 8

Target:

5

Answer:

Index 1

Need:

Stop at first match

Pattern:

Search

+

Early Exit

4. Last Occurrence

Example:

2 5 3 5 8

Answer:

Index 3

Need:

পুরো array check করতে হবে।

```
int index=-1;

for(int i=0;i<n;i++)
{
    if(arr[i]==target)
    {
        index=i;
    }
}
```

Chapter 4 — Search State

Search problem-এ সাধারণত ৩ ধরনের state লাগে।

1. Found State

Question:

Exists?

Use:

```
int found=0;
```

Example:

Duplicate আছে?

2. Index State

Question:

Where?

Use:

```
int index=-1;
```

Why -1?

কারণ:

Valid index:
0 থেকে শুরু

-1 মানে:

Not Found

3. Count State

Question:

How many?

Use:

```
int count=0;
```

Pattern Table

Question	State
Exists?	flag

Question	State
Position?	index
How many?	counter
Largest?	max
Smallest?	min

Chapter 5 — Sorted Data & Search

Sorting কেন important?

কারণ sorted data information দেয়।

Example:

Unsorted:

8 2 9 1 5

Target:

7

কিছু জানা যায় না।

Sorted:

1 2 5 8 9

এখন:

যদি middle দেখি:

5

Target:

7

তাহলে বুঝি:

Left side বাদ

কারণ সব ছোট।

এটাই Binary Search-এর idea।

Chapter 6 — Binary Search Mental Model

Binary Search কী?

Sorted array-তে search করার efficient method।

Idea:

Every step এ half data বাদ

Example:

Array:

1 3 5 7 9 11 13

Find:

11

Step 1:

Middle:

7

Compare:

$11 > 7$

তাই:

Left side বাদ

Remaining:

9 11 13

Step 2:

Middle:

11

Found.

Linear Search:

n elements

↓

n check

Complexity:

$O(n)$

Binary Search:

n elements

half

half

half

Complexity:

$O(\log n)$

Binary Search Condition

Binary Search-এর জন্য:

Array অবশ্যই sorted হতে হবে

না হলে:

Wrong Answer

Chapter 7 — Binary Search Implementation

Iterative Version

Template:

```
int binarySearch(int arr[], int n, int target)
{
    int left = 0;
    int right = n-1;

    while(left ≤ right)
    {
        int mid = left + (right-left)/2;

        if(arr[mid] == target)
        {
            return mid;
        }

        else if(arr[mid] < target)
        {
            left = mid + 1;
        }

        else
        {
            right = mid - 1;
        }
    }

    return -1;
}
```

Understanding Variables

left

Search শুরু:

0

right

Search শেষ:

n-1

mid

Middle position:

`left + (right-left)/2`

Why not:

`(left+right)/2`

?

Large value হলে overflow হতে পারে।

Safe:

`left+(right-left)/2`

Chapter 8 — Binary Search Patterns

Binary Search শুধু value খোঁজা না।

এর বড় idea:

Search on Answer

Pattern 1 — First Occurrence

Example:

1 2 2 2 5

Target:

2

Need:

First 2

Binary Search modify:

Found হলে:

```
answer=mid
```

```
left side search
```

Pattern 2 — Last Occurrence

Same:

```
1 2 2 2 5
```

Need:

```
Last 2
```

Found হলে:

```
answer=mid
```

```
right side search
```

Pattern 3 — Lower Bound

Meaning:

First position যেখানে:

```
value ≥ target
```

Example:

1 3 5 7 9

target:

6

Answer:

Index 3

কারণ:

$7 \geq 6$

Chapter 9 — Common Bugs & Edge Cases

Bug 1 — Unsorted Array-এ Binary Search

Wrong:

8 2 5 1 9

Binary search I

Result:

Wrong হতে পারে।

Bug 2 — Infinite Loop

Wrong:

```
while(left < right)
```

অনেক ক্ষেত্রে সমস্যা করতে পারে।

Basic template:

```
while(left ≤ right)
```

Bug 3 — Wrong Update

Found না হলে:

Target বড়:

Wrong:

```
left=mid;
```

Correct:

```
left=mid+1;
```

Target ছোট:

Correct:

```
right=mid-1;
```


Bug 4 — Empty Array

$n=0$

Handle করতে হবে।

Bug 5 — Duplicate Values

Example:

2 2 2 2

Normal search:

যেকোনো index দিতে পারে।

First/Last occurrence আলাদা problem।

Chapter 10 — Day 7 Assignment

Part A — Linear Search Practice

Implement:

1. Search value exists কিনা
2. Search index
3. Count occurrence
4. First occurrence
5. Last occurrence
6. Maximum value index
7. Minimum value index

Part B — Binary Search Practice

Implement:

1. Basic Binary Search
2. First occurrence
3. Last occurrence
4. Count frequency in sorted array
5. Lower Bound

Part C — Dry Run

Array:

1 3 5 7 9 11 13

Target:

11

Write:

left:

right:

mid:

arr[mid]:

decision:

প্রতিটি step।

Part D — Pattern Recognition

বলবে কোন pattern লাগবে:

Problem 1

একটি number array-তে আছে কিনা।

Answer:

?

Problem 2

Sorted array-তে first occurrence বের করো।

Answer:

?

Problem 3

একটি value কতবার এসেছে।

Answer:

?

Problem 4

Huge sorted array-তে search করতে হবে।

Answer:

?



Glossary Update

Add:

Search

Linear Search

Binary Search

Search Space

Target

Found State

Index State

Occurrence

First Occurrence

Last Occurrence

Lower Bound

Upper Bound

Mid

Left Pointer

Right Pointer

Sorted Data

Logarithmic Time



Pattern Library Update

Add:

Linear Search Pattern

Recognition:

Need to find value/index/count.

Mental Model:

Traverse

↓

Check

↓

Match?

↓

Update State

Complexity:

$O(n)$

Binary Search Pattern

Recognition:

- Data is sorted
- Need fast search

Mental Model:

Middle Check

↓

Discard Half

↓

Repeat

Complexity:

$O(\log n)$

Day 7 Completion Checklist

- ☐ Searching concept বুঝি
- ☐ Linear Search লিখতে পারি
- ☐ Found state ব্যবহার করতে পারি
- ☐ Index বের করতে পারি
- ☐ Count occurrence করতে পারি
- ☐ First occurrence বুঝি
- ☐ Last occurrence বুঝি
- ☐ Sorted data-এর advantage বুঝি
- ☐ Binary Search-এর idea বুঝি
- ☐ Binary Search implement করতে পারি
- ☐ left/right/mid বুঝি
- ☐ First occurrence binary search বুঝি
- ☐ Last occurrence binary search বুঝি
- ☐ Lower bound concept চিনেছি
- ☐ Edge cases handle করতে পারি

Day 7 Final Mental Model

Search Problem



What do I need?



Exist?

→ Flag

Position?

→ Index

Count?

→ Counter



Is data sorted?



No

→ Linear Search

Yes

→ Binary Search possible



Can I remove half?



Efficient Solution

Day 7 শেষে তোমার CP foundation এ নতুন layer যোগ হলো:

Day 1-5

Problem Patterns

+

Day 6

Code Organization

+

Day 7

Efficient Searching

পরের **Day 8 — Sorting Algorithms Deep Dive + Complexity Thinking** হবে।